

Simultaneous Path Planning and Topological Mapping (SP²ATM) for environment exploration and goal oriented navigation

Shuzhi Sam Ge^{a,*}, Qun Zhang^b, Aswin Thomas Abraham^b, Brice Rebsamen^c

^a Interactive & Digital Media Institute (IDMI), Department of Electrical and Computer Engineering, National University of Singapore, Singapore 117576

^b NUS Graduate School for Integrative Sciences and Engineering (NGS), National University of Singapore, Singapore 117456

^c Temasek Laboratories, National University of Singapore, Singapore 117411

ARTICLE INFO

Article history:

Received 31 May 2010

Received in revised form

11 October 2010

Accepted 11 December 2010

Available online 4 January 2011

Keywords:

Path planning

Hierarchical topological mapping

Map-based navigation

Autonomous mobile robots

ABSTRACT

In this paper, a novel approach, Simultaneous Path Planning and Topological Mapping (SP²ATM), is presented to address the problem of path planning in unknown environments by concurrent and incremental construction of a map, which strictly exploits only the topology rather than grid representation. For local topological information representation, a new concept, Admissible Space Tree (AST), is presented to describe the admissible free space in the environment as a group of nodes and graphs. The global map of the explored environment is encoded in a Hierarchical Topological Map (HTM), which by embedding the AST, serves as the least information to facilitate path planning. For simplicity, the algorithm is implemented in a planar space on our differentially driven mobile robot X1, based on its range sensing and self-localization capabilities. Experiments' results show that SP²ATM is effective and globally convergent in complex and dynamic environments.

© 2010 Elsevier B.V. All rights reserved.

1. Introduction

Path planning and map building are fundamental requirements for an autonomous mobile robot, enabling it to navigate from an initial configuration to a final one, in an environment unknown *a priori*. A recent and general study on this subject can be found in [1,2]. In the canonical *goal oriented path planning* or *exploratory path planning* problem, the full knowledge of the environment is assumed to be known. Solutions to this conventional problem cannot be directly employed to address the problem of sensor-based planning and mapping an unknown environment through systematic exploration [3]. As acquiring the world model, the robot needs to plan a path simultaneously. However, bringing forth a single solution to these problems in practice turns out to be difficult. First, an efficient representation of the environment must be modeled. Secondly, the representation built from imperfect sensor readings must cope with inaccurate localization. Finally, a fully autonomous robot must be able to simultaneously plan its motion while building the representation online. An active and related problem is Simultaneous Localization and Mapping (SLAM), the solution to which produces an accurate map from perceived information for robot localization [4], but does not address the problem of how to steer the robot in order to autonomously

complete the task [5]. For a robot which does not make use of any predefined waypoints, a path planning algorithm becomes inevitable while mapping an unknown space. A number of studies which are exclusively intended for planning exploration strategies for SLAM can be found in [5–7].

The literature is full of elegant solutions to the path planning problem such as the bug algorithm [8] and potential functions [9,10] without producing a map. Since a path can be generated more efficiently using a map [11], autonomous robots must be able to build maps of their environments during exploration. Two types of models for mapping environments in mobile robot navigation have been mainly produced [12]: grid based and topological. Although grid-based algorithms [13,14] are easy to build accurate metric maps, they suffer from inefficiencies of planning in time and space, especially when mapping a large scale space. Topological representation, therefore, becomes the essential approach to map building for an efficient and optimal path planning solution under low space requirements. Topological map generally takes the form of a graph, having nodes representing significant states or locations and edges representing trajectories or control actions between two different configurations [15]. Even though difficult to construct, it is immune to inaccurate localization and robot slippage [16] and provides improved flexibility over pure metric and hybrid metric topologies for dynamic planning.

The years have witnessed several contributions to topological path planning in unknown environments since its proposal in [17]

* Corresponding author. Tel.: +65 6516 6821; fax: +65 6779 1103.

E-mail address: samge@nus.edu.sg (S.S. Ge).

Nomenclature

$\mathbf{q}_r, \mathbf{q}_t$	Position of the robot and the final goal in the global frame;
$\Omega_{ws}, \delta_F, \delta_{AF}$	The robot's workspace, free space and admissible free space;
w_r, w_v	Actual and virtual base width of the robot;
γ, γ_h	Resolution and horizontal field of view of the range sensor;
$d(s_t), s_t$	Distance from the sensor frame origin and the fixed step size;
$d_{\max}, d_{\min}$	Maximum and minimum ranges of the sensor;
$\mathcal{W}$	A wavefront in a range scan;
$\mathcal{F}_j, \mathcal{F}$	A fork and set of all forks in a range scan;
$\mathcal{X}_j, \mathcal{X}$	A section and set of all sections in a range scan;
d_θ, θ	The range reading and inclination from the sensor frame y axis;
$\mathcal{C}, w_c, D_c, \theta_c$	A corridor, its width, length and inclination from the sensor frame y axis;
δ_{kj}, δ_k	The j th segment and set of segments in $\mathcal{A}_k$;
$\mathcal{L}_{kj}, \mathcal{L}_k$	A leaf and set of all leaves in a range scan;
$\mathcal{A}_k, \mathcal{A}_{\text{tot}}, \mathcal{A}$	The k th AST, total number of AST in $\mathcal{M}$ and set of all ASTs;
$\mathcal{M}, \mathcal{K}$	The HTM and the topology graph;
$\mathcal{T}_j, \mathbf{q}_{\mathcal{T}_j}, N_{\mathcal{T}}$	The j th topology node in $\mathcal{T}$, its coordinates in the global frame and the total number of nodes in $\mathcal{K}$;
$\mathcal{P}_{kj}, \mathcal{P}_k$	The j th possibility and set of all possibilities in $\mathcal{A}_k$;
$\mathcal{P}_{n\mathcal{T}_j}, \mathcal{P}_{\mathcal{T}_j}, N_{\mathcal{P}_{\mathcal{T}_j}}$	The n th possibility, set of all possibilities and the total number of possibilities assigned to $\mathcal{T}_j$;
$\mathcal{T}_r$	The topology node associated with the current position of the robot $\mathbf{q}_r$;
E_l, V_r, r_t	The type of $\mathcal{T}_j$, the vicinity circle and its radius;
$d(a, b)$	The Euclidean distance between any two points a and b in the global frame;
t_d, σ_g	The tolerance period and tolerance distance;
$d_{\mathcal{T}}$	Minimum distance between two nodes in the map;
$R_{i,j}, d_R(i, j)$	The shortest route and the route distance from $\mathcal{T}_i$ to $\mathcal{T}_j$ in $\mathcal{K}$;
$d_v(i, j)$	The visibility distance between two nodes $\mathcal{T}_i$ and $\mathcal{T}_j$ in $\mathcal{K}$;
$\mathcal{T}_s, \mathcal{T}_v(i, j)$	Starting node and the visibility node in $R_{i,j}$;
$\mathcal{I}_g, \mathbf{q}_i$	Instant goal generated by the planner and its position in the global frame.

where a high level topological map is proposed by linking the distinctive places and travel edges. Exploration strategies are presented in [18,19] for the robot to explore unknown environments which are modeled as a graph. The *sightseer strategy* and the *seed spreader strategy* are proposed in [20] to approach the exploratory path planning problem in planar terrain randomly populated with obstacles of arbitrary shapes. *Coverage path planning* employed for applications such as floor cleaning and robotic demining is surveyed in [11]. A fully integrated reactive system is proposed in [21], where a topological map is maintained to minimize the amount of stored information and aid path planning. A high level map known as the Enhanced Topological Map, which integrates rough metrical information is proposed in [22] for reliable localization and navigation. A navigation graph in [23] is built to plan paths among stationary polygonal obstacles. Another solution to motion planning problem is approached in [24] by embedding a dynamically maintained single-source shortest path tree into hierarchical approximate cell decomposition. A great contribution to the sensor-based motion planning [3] is that of the Generalized Voronoi Graph (GVG) [25], whose incremental

construction of unknown environments is shown possible in [26] by proposing several control laws which direct the robot to steer in a specific manner. A Next Best View (NBV) algorithm is proposed in [5] for safe and efficient map building by introducing a new concept of a safe region. A Gap Navigation Tree (GNT) is presented in [27] for distance-optimal navigation in unknown environments by employing only a gap sensor. Most of the aforementioned seminal approaches solve the problem by employing topology only at a high level, while a few others, even though globally optimal in path traveled, consumes a considerable amount of time during incremental mapping.

Mobile robot path planning can be categorized as global planning and local planning. The global path planning algorithms build a world model based on local sensory information and ensure global convergence, while the local path planning approaches utilize local sensory data in a reactive manner to guide the robot through a collision-free path to a visible configuration provided by the global planner. The approach presented in [28] deals with the problem of reactive collision avoidance in very dense, cluttered, and complex scenarios. A situated-activity paradigm is introduced to design the reactive navigation system of a circular robot by utilizing the Nearness Diagram (ND) to represent the environment. In [29] a generalized space transformation framework for reactive navigation of a kinematically constrained robot is proposed. In this approach, the robot of arbitrary shape is transformed into a point in the trajectory parameter space by the Parameterized Trajectory Generators (PTGs) for easier collision avoidance of non-holonomic robots. However, as most reactive navigation methods cannot guarantee the global convergence and the classification of global and local planning always makes a navigation system complex, the situation becoming more difficult when the robot maneuverability has to be taken into consideration, a single layered path planner which at one time performs the dual function of both global and local planners is therefore needed to increase the efficiency for fast and smooth path planning in cluttered and dynamic environments. In addition, to improve flexibility in representation, it is also desirable to arrange the topological maps in a hierarchical manner such as in [17,30] by utilizing only the minimum information necessary to complete the task [27].

The single layered, optimal and complete Simultaneous Path Planning and Topological Mapping (SP²ATM) algorithm is presented in this paper, which only utilizes a Hierarchical Topological Map (HTM), incrementally constructed online without any preprocessing or query stages, and carries out the path planning simultaneously based on the map built. Realistic range sensors termed as *extended range detectors* which provide imperfect range readings larger than the robot, but less than infinite, are employed [30]. Unlike many map building algorithms which model the obstacles in the scene, our focus is restricted to extracting the *admissible free space* in the environment, leading to the construction of the *Admissible Space Tree* (AST). A set of instant goals [31] are selected from critical points in the AST, known as possibilities, to guide the robot across the space, which increase the likelihood of obtaining an improved global performance for every increase in the sensor detection range. The HTM represents the global topology of the environment by an undirected graph with the AST embedded in its nodes. The resulting SP²ATM algorithm could be employed to reach a given goal within the accessible space (goal oriented path planning) as well as to explore all the accessible space in an unknown environment (exploratory path planning), both of which are presented in this paper. The planner could be classified under global path planning, but does not require modules for local planning and obstacle avoidance. Moreover, the low level motion control is completely separated and the map building does not require the robot to move along a specific path such as in the construction of many

roadmaps. This not only promotes uninterrupted and smooth motion, but also contributes to task completion in minimal time. However, to produce a complete navigation module, we do assume that the robot is equipped with a simple relative localization system (e.g. dead reckoning). It will be shown that the robot is capable of performing reliable navigation by assuming that the localization drift is bounded. Optimal motion is made possible by guiding the robot to move towards a set of instant goals such as in [31], and through continuous replanning which prevents the robot from always focussing on a single region. This produces a natural human-like behavior by instantly moving towards newly perceived regions in the environment, if they are of greater interest.

The main contributions of this paper are:

- (i) A single layered and complete SP²ATM algorithm which produces an optimal solution for goal oriented path planning and exploratory path planning, while the HTM of an unknown, unstructured environment is simultaneously constructed, is presented.
- (ii) The admissible free space in the present sensor view is modeled by a set of nodes and graphs, which leads to the new concept of an Admissible Space Tree (AST). Possibilities, reflecting the possible direction to explore, are generated from critical points of the AST and employed for planning the next instant goal and updating the global topological map.
- (iii) A Hierarchical Topological Map (HTM) is proposed as a novel structure for representing the topology of an environment. The map which is incrementally built by maintaining a set of possibilities at each increase in the sensor detection range serves as the least information for path planning in a single configuration space.

In Section 2, the concepts of local topology extraction for AST construction is presented. The possibility and Hierarchical Topological Map (HTM) are described in Section 3. The SP²ATM algorithm and incremental construction of HTM are presented in Section 4. Experimental results are given in Section 5, followed by conclusions in Section 6.

2. Local topology representation: the Admissible Space Tree (AST)

One of the challenges in enabling a mobile robot navigate autonomously, is to construct a representation of the environment that can be used by the robot. Since we are faced with the problem of sensor-based incremental mapping, all information perceived is useful and must be stored in an efficient manner. As the first step to solving the problem, it makes sense to restrict the map building to extracting the *free space* available in the environment, since the main aim is to enable the robot to traverse through this free space.

Before proceeding further, the following assumptions are made.

Assumption 1. The robot operates in a subset Ω_{ws} of Euclidean space known as the workspace populated by n obstacles $C_1, \dots, C_n$ which are all closed sets.

Assumption 2. The robot is capable of estimating its position q_r in the global navigation frame with bounded drift from ground truth.

Assumption 3. The position of the goal q_g , stationary or moving, is known at any instant for goal oriented path planning in the global navigation frame.

The robot in the global frame and range sensor in the robot frame are shown in Figs. 1 and 2, respectively. Since a point robot is not assumed, it is not capable of moving through all the *free space* $\mathcal{F}$ in Ω_{ws} as shown in Fig. 3. It is necessary to specify the admissible free space $\mathcal{F}_{AF}$ in Ω_{ws} which ensures complete maneuverability of the robot. For the differentially driven rectangular robot which is required only to rotate over its current position and move straight, a virtual base width w_v ($w_v > w_r$) is assumed.

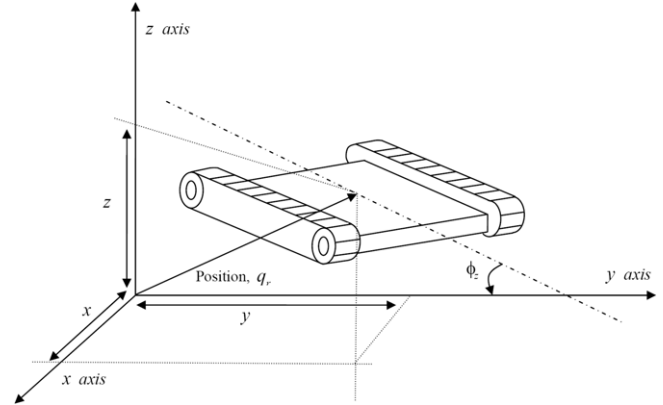


Fig. 1. Position of the robot q_r in the global frame.

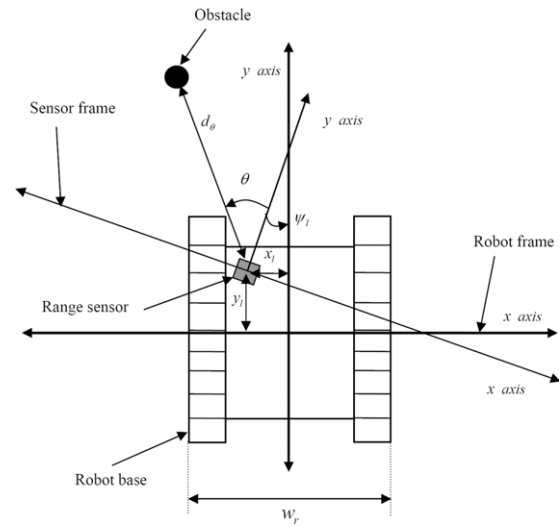


Fig. 2. Position of the range sensor in the robot frame, d_θ and θ in the sensor frame.

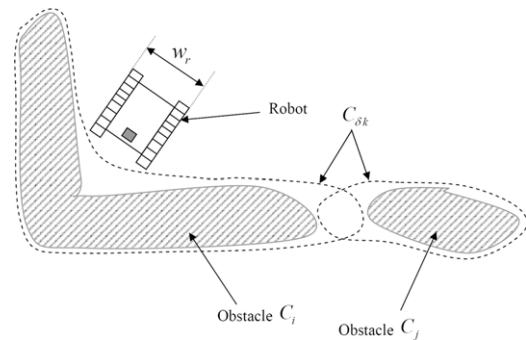


Fig. 3. Increased influence of an obstacle.

The obstacles C_j have a greater area of influence giving rise to a set of m ($n \neq m$) virtual obstacles $C_{\delta i}$ which are also closed sets. For ease of understanding, let us assume that a two-dimensional range scan is available. Portions of Ω_{ws} are analyzed to obtain the admissible free space $\mathcal{F}_{AF}$, which can be defined as portion of the environment that is traversable by the robot.

$$\mathcal{F}_{AF} = \Omega_{ws} \setminus \bigcup_{i=1}^{i=m} C_{\delta i}. \quad (1)$$

In this paper, it is assumed that the admissible free space is connected. Recording down the admissible free space can be

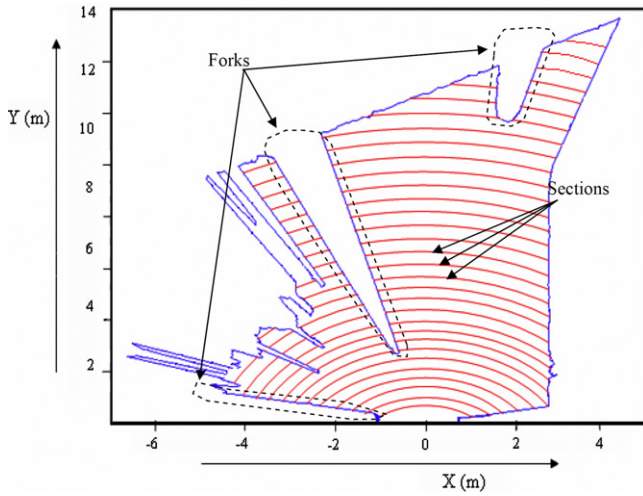


Fig. 4. Sections produced due to wavefront propagation.

done in a tree format which leads us to the construction of the Admissible Space Tree (AST).

2.1. Separating forks and sections

The distance information in a range scan provides one with the opportunity of dividing the perceived space into *free spaces* and *non-free spaces*. As discussed, a *free space* does not signify that it is completely traversable when taking into account the dimensions of the robot. In this section, we start describing the *free spaces* by a set of sections and *non-free space* including obstacles and unknown areas by a set of forks. The range scan analysis begins by propagation of a wavefront.

Definition 1 (Wavefront). A wavefront $\mathcal{W}$ is an imaginary arc spanning over the entire horizontal field of view γ_h at a distance $d(s_t)$ from the sensor frame origin, where s_t is a fixed step size for the wavefront propagation.

A wavefront is propagated from the minimum range $d(s_t) = d_{\min}$ to the maximum range $d(s_t) = d_{\max}$ of the range sensor, where $d_{\min} > 0$, otherwise a wavefront does not exist. The propagation of the wavefront is executed in steps of fixed length s_t . When it is completed, the wavefront is subdivided by forks to form sections, as shown in Fig. 4.

Definition 2 (Fork). A fork $\mathcal{F}_j \in \mathcal{F}$ is a portion in the range scan which obstructs a wavefront. If there are no obstacles in the range scan, only a single fork exists over the entire region uncovered by the horizontal field of view γ_h of the sensor. The forks are shown blue with three of them marked in Fig. 4.

Definition 3 (Section). A section $\mathcal{X}_j \in \mathcal{X}$ is a portion split by forks from a wavefront satisfying the condition $d(s_t) \leq d_\theta$, where d_θ is the range reading at inclination θ from the sensor frame y axis. A section is always formed in between two forks and $d(s_t) \leq d_\theta$ makes it clear that there are no obstacles along the section from the sensor frame origin.

An illustration of a set of sections are shown red with three of them marked in Fig. 4. The analysis to extract sections begins at the distance $d(s_t) = d_{\min}$ where the wavefront is drawn about the origin in the anti-clockwise direction, starting from the positive x axis on the sensor frame. All the range points d_θ in the scan are compared with the radius of the wavefront $d(s_t)$ to find the forks and the sections between them. The wavefront then moves forward by a step s_t to repeat the process.

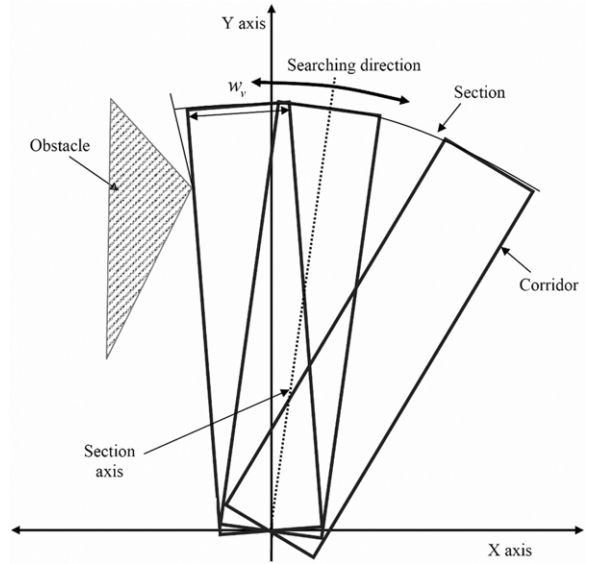


Fig. 5. Set of corridors searching a segment.

During analysis if $d_\theta > d(s_t)$ ($-\frac{\gamma_h}{2} \leq \theta \leq \frac{\gamma_h}{2}$), it denotes the start of a section if either $\theta = \frac{\gamma_h}{2}$ or $d_{(\theta-\gamma)} < d(s_t)$, where γ is the resolution of the range scanner. Any section being examined is complete when either $\theta = -\frac{\gamma_h}{2}$ or if $d_{(\theta+\gamma)} < d(s_t)$. Once a section is extracted from the wavefront, only those whose length is greater than robot width w_r are considered for further extracting segments. If there are no obstacles, only a single fork is found which gives rise to a single section $\mathcal{X}_j$ along the whole span of γ_h .

2.2. Extracting segments

At present we have the fundamental component sections that describe the *free space* in the environment as perceived by the range sensor. The next aim is to derive admissible free spaces in the environment. Such a space is described by a set of segments, which is useful for non-point robot path planning. The robot's maneuverability through the resulting *free space* is verified through a set of corridors.

Definition 4 (Corridor). A corridor $\mathcal{C}$ is a rectangle of width w_c , length D_c at θ_c orientation from the sensor frame y axis. It never encloses any obstacles in its area $w_c D_c$ and always rotates about the midpoint of its breadth $0.5w_c$ placed at the origin of the sensor frame.

Since there are no non-traversable regions enclosed, the existence of a corridor between the robot and a point implies that the point can be “seen” by the robot and is directly reachable from the current position. The admissible free space is then computed dynamically from available sensory data by rotating straight corridors over the origin of the sensor frame. Since the corridor width w_c is set according to the robot width w_r , the robot's dimensions are directly taken into consideration, which also implies that not all points in a section can be covered by rotating a corridor about the sensor frame origin.

Definition 5 (Segment). A segment $\mathcal{S}_{kj} \in \mathcal{S}_k$ is a portion of a section for which a set of corridors exist, i.e. $D_c = d(s_t)$. It represents a subset of admissible free space and the robot can traverse to any point on a segment from the current position.

Segments are extracted from sections. As shown in Fig. 5, in order to extract a segment from a section $\mathcal{X}_j$, range readings are examined from the center of the section to check if a corridor

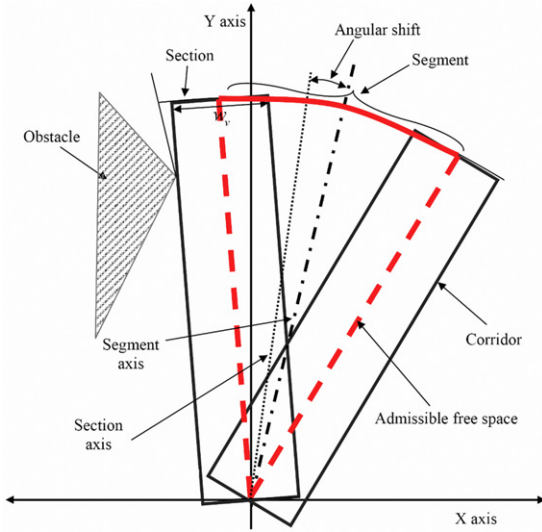


Fig. 6. Resulting segment and admissible free space (area outlined red and thick). (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

of width w_v exists at each point in both directions and it stops on either side of the section when no corridor exists. Since it is likely that numerous segments could be produced in a single propagation, priority for analysis is given to the section with the least inclination with the y axis. Analysis continues recursively for each section over all the wavefront propagations until $d(s_t) = d_{\max}$ and no segments exist for the entire γ_h .

A few such corridors along a section are shown in Fig. 5. All the section points verified eligible by a corridor combine to form the segment. In Fig. 6, the axis of the resulting segment can be seen to be shifted away from that of the section to the right. It can be seen that the resulting segment may not only be smaller but also asymmetric about the section it is derived from. The entire space from the sensor frame origin to this segment now describes a subset of admissible free space $\mathcal{A}_F$ (Fig. 6). The resulting subsets of admissible free spaces are denoted by nodes in a tree, leading to the construction of the AST.

2.3. Admissible Space Tree (AST)

The set of segments described by the same pair of forks represent a single admissible free space. Since the furthest information in any single admissible free space is the most useful for robot path planning, the segment which is the furthest away from the sensor frame origin between the same pair of forks is chosen and serves as a node of the k th AST $\mathcal{A}_k$ to represent the single admissible free space. As the analysis proceeds outwards, the tree grows in size to complete the $\mathcal{A}_k$ construction. Unlike all other nodes in $\mathcal{A}_k$, the root node $\mathcal{S}_R$ (the sensor frame origin) does not describe any area since it is just the start point of the tree. The end nodes in $\mathcal{A}_k$ known as leaves are the most useful to generate an instant goal for path planning.

Definition 6 (Leaf). A leaf $\mathcal{L}_{kj} \in \mathcal{S}_k$, is the furthest segment in any direction of propagation in $\mathcal{A}_k$, i.e. there exists no admissible free space after a leaf within the scan. It is of great importance for path planning since they are the admissible portions in the scan which extend outwards the most within the reach of the range sensor.

Fig. 7 depicts the complete AST and can be expressed as

$$\mathcal{A}_k = \{\mathcal{S}_k, \mathcal{L}_{\mathcal{A}_k}\}; \quad 0 < k \leq A_{\text{tot}} \quad (2)$$

where $\mathcal{L}_{\mathcal{A}_k(i,j)} \in \mathcal{L}_{\mathcal{A}_k}$ is the branch joining parent node $\mathcal{S}_{ki}$ to the child node $\mathcal{S}_{kj}$. It can be clearly seen that a node $\mathcal{S}_{kj}$ represents

an admissible free space and the robot can now traverse to any point in the space. As a compact representation of admissible free spaces in the two-dimensional world, the AST may not represent the local topology in great detail such as in grid mapping, but is just sufficient for path planning.

3. Possibilities and the Hierarchical Topology Map (HTM)

The Hierarchical Topological Map (HTM) denoted by $\mathcal{M}$ is a two-layered structure which illustrates the general topology of Ω_{ws} with AST $\mathcal{A}$ in its lower layer and an undirected graph $\mathcal{K}$ in the upper layer. The whole HTM can therefore be expressed as

$$\mathcal{M} = \{\mathcal{K}, \mathcal{A}\}. \quad (3)$$

Topology graph $\mathcal{K}$ contains a set of nodes connected by edges expressed as

$$\mathcal{K} = \{\mathcal{T}, \mathcal{L}_{\mathcal{K}}\} \quad (4)$$

where $\mathcal{T}_j \in \mathcal{T}$ is the j th topology node and $\mathcal{L}_{\mathcal{K}(i,j)} \in \mathcal{L}_{\mathcal{K}}$ is the edge between $\mathcal{T}_j$ and an i th topology node $\mathcal{T}_i$. It depicts the overall topology of the environment with each node describing AST perceived from its location. This could result in partial overlap between $\mathcal{A}_k$ and $\mathcal{A}_j$ ($k \neq j$), but does not affect path planning (see Section 3.2). In theory, since $\mathcal{A} \subseteq \mathcal{K}$, the knowledge representation of $\mathcal{M}$ can also be expressed in terms of the incidence matrix I_G .

$$I_G[v, e] = \begin{cases} 0 & \text{if } v \text{ is not an endpoint of } e \\ 1 & \text{if } v \text{ is an endpoint of } e \\ 2 & \text{if } e \text{ is a self-loop at } v \end{cases} \quad (5)$$

where $v \in \{\mathcal{T}, \mathcal{S}_k\}$, $0 < k \leq A_{\text{tot}}$ and $e \in \{\mathcal{L}_{\mathcal{K}}, \mathcal{L}_{\mathcal{A}}\}$.

The layers in the HTM are shown in Fig. 8. In order to construct the HTM, the robot is allowed to move towards an instant goal at any instant of time. These instant goals are selected from possibilities, critical points in AST $\mathcal{A}$. The constructed HTM and the available possibilities at any instant of time further help the robot with path planning.

3.1. Possibility

Definition 7 (Possibility). A possibility $\mathcal{P}_{kj} \in \mathcal{P}_k$ is a possible waypoint in the global navigation frame chosen from a leaf segment $\mathcal{L}_{kj}$ in $\mathcal{A}_k$ for aiding path planning.

Definition 8 (Topology Node). A topology node is denoted as $\mathcal{T}_j = (\mathbf{q}_{\mathcal{T}_j}, \mathcal{P}_{\mathcal{T}_j}, E_l, V_r)$, $l = 1, 2$. The four elements characterizing a topology node are described as follows:

- (i) $\mathbf{q}_{\mathcal{T}_j}$ (Node position): the node coordinates in the global navigation frame.
- (ii) $\mathcal{P}_{\mathcal{T}_j}$ (Possibility): a set of all possibilities assigned to $\mathcal{T}_j$, $\mathcal{P}_{\mathcal{T}_j} = \{\mathcal{P}_{1\mathcal{T}_j}, \mathcal{P}_{2\mathcal{T}_j}, \dots, \mathcal{P}_{n\mathcal{T}_j}\}$.
- (iii) E_l (Type): the node type depicts the characteristic of the node.
 1. E_1 (Explored): a node which has no possibilities, i.e. $\mathcal{P}_{\mathcal{T}_j} = \emptyset$.
 2. E_2 (Unexplored): a node having at least one possibility, i.e. $\mathcal{P}_{\mathcal{T}_j} \neq \emptyset$.
- (iv) V_r (Vicinity circle): an imaginary circle of radius r_t represents the region of coverage of a node in the map.

In this paper, the possibilities are generated from the leaf segments according to different applications. For exploratory path planning, the center of the leaf segment is assigned as the j th possibility using the midpoint formula. In goal oriented path planning, the ends of a leaf segment are the most useful portions since it would give rise to efficient obstacle avoidance towards the goal. The segment end which is closer to the final goal $\mathbf{q}_f$ is chosen using the Euclidean distance formula. The coordinates of the selected possibilities in the AST are then transformed into the global navigation frame.

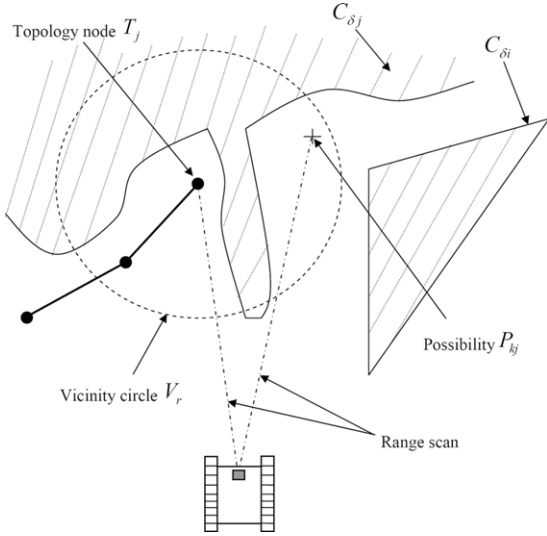


Fig. 9. Possibility in node vicinity problem.

where $d(\mathcal{P}_{n\mathcal{T}_j}, \mathcal{P}_{k_j})$ is the Euclidean distance between $\mathcal{P}_{n\mathcal{T}_j}$ and $\mathcal{P}_{k_j}$, σ_p is the matching tolerance between any two possibilities, which determines the degree of overlapping between admissible free spaces. If the above condition fails, then the new possibility is rejected, i.e. $\mathcal{P}_k = \mathcal{P}_k \setminus \mathcal{P}_{k_j}$.

3.2.4. Rearranging pre-assigned possibilities

During map building, it is necessary to revise the existing set of possibilities in $\mathcal{K}$ for optimal coverage. For assigned possibilities $\mathcal{P}_{\mathcal{T}_j}$, $j \neq r$, which would be closer to the node $\mathcal{T}_r$ than $\mathcal{T}_j$,

$$d(\mathcal{P}_{n\mathcal{T}_j}, \mathbf{q}_{\mathcal{T}_r}) < d(\mathcal{P}_{n\mathcal{T}_j}, \mathbf{q}_{\mathcal{T}_j}) \quad (10)$$

where $d(\mathcal{P}_{n\mathcal{T}_j}, \mathbf{q}_{\mathcal{T}_r})$ is the Euclidean distance between $\mathcal{P}_{n\mathcal{T}_j}$ and $\mathbf{q}_{\mathcal{T}_r}$, and $d(\mathcal{P}_{n\mathcal{T}_j}, \mathbf{q}_{\mathcal{T}_j})$ is the Euclidean distance between $\mathcal{P}_{n\mathcal{T}_j}$ and $\mathcal{T}_j$, it is fair to reassign them to the new node $\mathcal{T}_r$ if they are directly visible from it. The visibility of the possibility $\mathcal{P}_{n\mathcal{T}_j}$ is checked from the present robot location $\mathbf{q}_r$ by verifying the existence of a corridor.

$$\text{if } \mathcal{C}(w_r, \theta_{\mathcal{T}_j}, d(\mathbf{q}_r, \mathcal{P}_{n\mathcal{T}_j})) \text{ exists, then } \mathcal{P}_{\mathcal{T}_j} = \mathcal{P}_{\mathcal{T}_j} \cup \mathcal{P}_{n\mathcal{T}_j}, \quad (11)$$

where $\mathcal{C}(w_r, \theta_{\mathcal{T}_j}, d(\mathbf{q}_r, \mathcal{P}_{n\mathcal{T}_j}))$ is the corridor between $\mathbf{q}_r$ and $\mathcal{P}_{n\mathcal{T}_j}$.

3.2.5. Assigned possibilities close to the robot

The set of associated possibilities $\mathcal{P}_{\mathcal{T}_j}$ for all j provide directions for the robot. When the robot moves close to $\mathcal{P}_{\mathcal{T}_j}$, they provide no more information for the planner and are set to be of inactive type. In the case that the map of the explored area is not required, these possibilities can be directly rejected. The rejection is executed if and only if corridor check is valid.

$$\text{If } d(\mathbf{q}_r, \mathcal{P}_{n\mathcal{T}_j}) < \lambda_{pr} \text{ and } \mathcal{C}(w_r, \theta_{\mathcal{P}_{n\mathcal{T}_j}}, d(\mathbf{q}_r, \mathcal{P}_{n\mathcal{T}_j})) \text{ exists, then } \mathcal{P}_{\mathcal{T}_j} = \mathcal{P}_{\mathcal{T}_j} \setminus \mathcal{P}_{n\mathcal{T}_j} \quad (12)$$

where $d(\mathbf{q}_r, \mathcal{P}_{n\mathcal{T}_j})$ is the Euclidean distance between $\mathbf{q}_r$ and $\mathcal{P}_{n\mathcal{T}_j}$, λ_{pr} is the robot possibility rejection distance, and $\mathcal{C}(w_r, \theta_{\mathcal{P}_{n\mathcal{T}_j}}, d(\mathbf{q}_r, \mathcal{P}_{n\mathcal{T}_j}))$ is the corridor between $\mathbf{q}_r$ and $\mathcal{P}_{n\mathcal{T}_j}$.

3.3. Updating the HTM

The HTM is incrementally updated as the robot moves when new admissible free space is perceived. While building $\mathcal{M}$, the topology node $\mathcal{T}_r$ to which the present robot position $\mathbf{q}_r$ is associated must be known at any instant of time. Since the vicinity

circle V_r of a set of topology nodes overlap with each other, the present robot position $\mathbf{q}_r$ could lie in more than one such circle, ensuring that the robot does not lose track of its position in $\mathcal{M}$. From the overlapping set of nodes, $\mathcal{T}_r$ is assigned to be the one closest to $\mathbf{q}_r$.

If the total number of nodes $N_{\mathcal{T}} = 0$ or $\mathcal{T}_r$ is not obtained, i.e. the robot has moved into a new region in Ω_{ws} , the new node $\mathcal{T}_1$ is registered as the starting node $\mathcal{T}_s$ in Ω_{ws} and becomes the current associated node $\mathcal{T}_r$ to the robot. $\mathcal{A}_1$ perceived at the starting node with the new possibility set $\mathcal{P}_1 \neq \emptyset$ after rejection process are directly assigned to $\mathcal{T}_1$. After that as the robot continues to explore the environment, new node $\mathcal{T}_j$ is registered in $\mathcal{M}$ and connected to $\mathcal{T}_r = \mathcal{T}_{j-1}$ (the robot associated node registered in the previous iteration) by an edge $L_{\mathcal{K}(j-1,j)}$ if a new $\mathcal{A}_k$ is obtained, i.e. there is at least a single admissible space (possibility) perceived but unregistered. $\mathcal{T}_j$ is then assigned to $\mathcal{T}_r$. In this way, the topology nodes $\mathcal{T}$ are appended one by one into $\mathcal{M}$ concurrently during robot motion, and all nodes $\mathcal{T}_j$ with possibility status $\mathcal{P}_{\mathcal{T}_j} = \emptyset$ and $\mathcal{P}_{\mathcal{T}_j} \neq \emptyset$ are updated to be of types E_1 and E_2 , respectively.

It is also to be made sure that there exists a minimum distance $d_{\mathcal{T}}$ in between the nodes. This is done so that the nodes are not so close to each other and does not cause any problem for the motion controller. Therefore the condition $d(\mathbf{q}_{\mathcal{T}_i}, \mathbf{q}_{\mathcal{T}_j}) \leq d_{\mathcal{T}}$, $i \neq j$ overrides the regular node registration. All the possibilities perceived while the robot is still within the distance of $d_{\mathcal{T}}$, will be assigned to the same node $\mathcal{T}_r$.

In unknown environment navigation, the robot always need to traverse back through an already mapped space Ω_{ws} by following the topology nodes in $\mathcal{K}$ along a designed route.

Definition 9 (Route). A route $R_{i,j}$ is the shortest path in $\mathcal{K}$ from $\mathcal{T}_i$ to $\mathcal{T}_j$.

The Bellman–Ford algorithm [32] is applied to find the route distance $d_R(i, j)$ of the route $R_{i,j}$. The Bellman–Ford equation is given as:

$$d_R(i, j) = \min_{k \in \text{neighbors}} \{d_R(i, k) + d_R(k, j)\}. \quad (13)$$

Definition 10 (Visibility Distance). The visibility distance denoted by $d_v(i, j)$ is the Euclidean distance from $\mathbf{q}_r$ to the farthest visible node $\mathcal{T}_v(i, j)$ in the route $R_{i,j}$. $\mathcal{T}_v(i, j)$ is termed as the visibility node in the route and satisfies

$$d(\mathbf{q}_r, \mathbf{q}_{\mathcal{T}_v(i,j)}) = \max_{r \leq k \leq j} \{d(\mathbf{q}_r, \mathbf{q}_{\mathcal{T}_k})\} \text{ and } \mathcal{C}(w_v, \theta_{\mathcal{T}_v(i,j)}, d(\mathbf{q}_r, \mathbf{q}_{\mathcal{T}_v(i,j)})) \text{ exists.} \quad (14)$$

A new edge will be added in between the present node $\mathcal{T}_r$ and any other node $\mathcal{T}_j$ ($\mathcal{T}_r$ and $\mathcal{T}_j$ are neither the same node nor neighbors) if it satisfies the following:

- (i) There exists a corridor $\mathcal{C}(w_r, \theta_{\mathcal{T}_j}, d(\mathbf{q}_r, \mathbf{q}_{\mathcal{T}_j}))$ to the node $\mathcal{T}_j$ from $\mathbf{q}_r$.
- (ii) The cost of travel from $\mathcal{T}_r$ to $\mathcal{T}_j$ is large, i.e. $d(\mathbf{q}_r, \mathbf{q}_{\mathcal{T}_j}) < \frac{d_R(r,j)}{\lambda_l}$, where λ_l is the route cost scale down factor.

A partial $\mathcal{K}$ built is shown in Fig. 10 which contains a set of nodes $\mathcal{T}$ connected by edges $L_{\mathcal{K}}$. The set of possibilities assigned to each node are shown as representing the admissible free spaces. The bridge which connects any node $\mathcal{T}_j$ to a possibility $\mathcal{P}_{n\mathcal{T}_j}$ is also shown as dotted lines projected out. The node $\mathcal{T}_r$ associated with $\mathbf{q}_r$ is shown. It can be seen that nodes $\mathcal{T}_1$ and $\mathcal{T}_2$ have the minimum distance $d_{\mathcal{T}}$ between them since they perceive new areas in the environment as the robot moves. This is necessary since there is only topological guidance for the robot and the nodes must be close enough to ensure proper accessibility to the perceived areas.

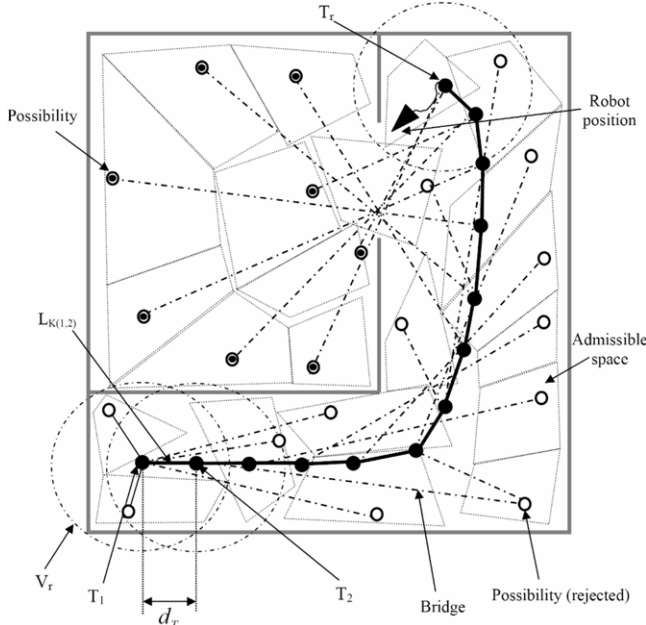


Fig. 10. A partially built topology graph.

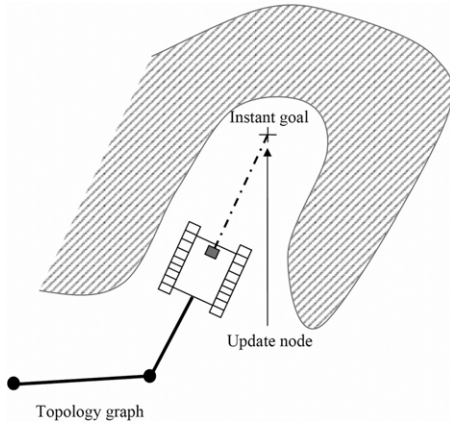


Fig. 11. Special case in topology node updating.

4. SP²ATM by incremental construction of the HTM

Possibilities perceived are most essential for path planning and leads to the creation of a HTM by utilizing leaves (as mentioned in Definition 6). The HTM is directly constructed by considering leaves from $\mathcal{A}$ for possibility generation. In order to reduce memory utilized, the HTM only contains possibilities instead of complete ASTs, reducing $\mathcal{M}$ to $\mathcal{K}$ on mission completion. The algorithm generates a new instant goal $\mathcal{I}_g \in \bigcup_{k=1}^{k=A_{tot}} \mathcal{P}_k$ or $\bigcup_{n=1}^{n=N_r} \mathcal{P}_{T_n}$, which is an intermediate point $\mathbf{q}_i$ in Ω_{ws} towards which the robot steers, for every sampling time. The planner follows a state machine framework $\mathcal{G} = \{\mathcal{G}_{Brake}, \mathcal{G}_{GoalApp}, \mathcal{G}_{Exp}, \mathcal{G}_{TraceBack}\}$, and exists only in a single state $\mathcal{G}_r$ at any instant of time. The different states are described below.

- (i) $\mathcal{G}_{Brake}$ (Braked mode): The planner is in this state before starting and after completion of the mission and does not provide any new $\mathcal{I}_g$ for robot motion. However, the robot may continue its state of motion towards a previous $\mathcal{I}_g$, if one was provided before entering this state and the robot is to be halted by the motion controller if required.
- (ii) $\mathcal{G}_{GoalApp}$ (Goal Approaching mode): The robot moves in pursuit of a goal positioned at $\mathbf{q}_t$ in the global frame to execute goal oriented path planning.

- (iii) $\mathcal{G}_{Exp}$ (Exploration mode): The planner aims to visit all admissible spaces $\mathcal{S}_{AF}$ to complete exploratory path planning.
- (iv) $\mathcal{G}_{TraceBack}$ (Trace Back mode): The planner switches to this state when the robot has to come back to a previously known node $\mathcal{T}_j$ in $\mathcal{K}$.

SP²ATM is executed in a sequential manner where $\mathcal{G}_r$ changes with time. Initially the planner is in *Braked* mode ($\mathcal{G}_r = \mathcal{G}_{Brake}$) and external input is provided to start or shift between $\mathcal{G}_{GoalApp}$, $\mathcal{G}_{Exp}$ and $\mathcal{G}_{TraceBack}$. σ_g is the tolerance in association between $\mathbf{q}_r$ and $\mathbf{q}_t$. The overall execution process is shown in Algorithm 1.

Algorithm 1 SP²ATM algorithm

```

1:  $\mathcal{G}_r = \mathcal{G}_{Brake}$ 
2: if  $\mathcal{G}_r \leftarrow \mathcal{G}_{GoalApp}$  then
3:   Execute GoalApproaching and update  $\mathcal{K}$  in  $\mathcal{M}$ 
4:   if  $\mathcal{G}_r \leftarrow \mathcal{G}_{TraceBack}$  then
5:     Execute TraceBack,  $\mathcal{G}_r \leftarrow \mathcal{G}_{GoalApp}$  and GOTO Step 2
6:   end if
7: end if
8: if  $\mathcal{G}_r \leftarrow \mathcal{G}_{Exp}$  then
9:   Execute Exploration and update  $\mathcal{K}$  in  $\mathcal{M}$ 
10:  if  $\mathcal{G}_r \leftarrow \mathcal{G}_{TraceBack}$  then
11:    Execute TraceBack,  $\mathcal{G}_r \leftarrow \mathcal{G}_{Exp}$  and GOTO Step 8
12:  end if
13: end if
14:  $\mathcal{G}_r \leftarrow \mathcal{G}_{Brake}$ 
15: EXIT

```

4.1. Goal approaching mode

The path planner provides instant goals $\mathcal{I}_g$ enabling the robot to move along a path which leads to the goal at $\mathbf{q}_t$. In Algorithm 2, $d(\mathbf{q}_r, \mathbf{q}_t)$ the Euclidean distance between $\mathbf{q}_r$ and $\mathbf{q}_t$, $d(\mathbf{q}_r, \mathbf{q}_{\mathcal{T}_v(i,j)})$ is the Euclidean distance between $\mathbf{q}_r$ and the visibility node $\mathcal{T}_v(i, j)$, $d_R(v, j)$ is the route distance of the route $R_{v,j}$, $d(\mathbf{q}_{\mathcal{T}_j}, \mathcal{P}_{n\mathcal{T}_j})$ is the Euclidean distance between $\mathbf{q}_{\mathcal{T}_j}$ and $\mathcal{P}_{n\mathcal{T}_j}$, and $d(\mathcal{P}_{n\mathcal{T}_j}, \mathbf{q}_t)$ is the Euclidean distance between $\mathcal{P}_{n\mathcal{T}_j}$ and $\mathbf{q}_t$.

Algorithm 2 Goal approaching mode

```

1: if  $d(\mathbf{q}_r, \mathbf{q}_t) \leq \sigma_g$  then
2:   Goal reached,  $\mathcal{G}_r \leftarrow \mathcal{G}_{Brake}$ , EXIT
3: end if
4: if  $\mathcal{C}(w_r, \theta_t, d(\mathbf{q}_r, \mathbf{q}_t))$  exists then
5:    $\mathbf{q}_i \leftarrow \mathbf{q}_t$ , GOTO Step 1
6: end if
7: Search the possibility  $\mathcal{P}_{n\mathcal{T}_j}$  closest to the goal from all the existing possibilities with the following path length equation minimized

$$\min_{0 < j \leq N_r} \left\{ \min_{0 < n \leq N_{\mathcal{P}_{\mathcal{T}_j}}} \{d(\mathbf{q}_r, \mathbf{q}_{\mathcal{T}_v(i,j)}) + d_R(v, j) + d(\mathbf{q}_{\mathcal{T}_j}, \mathcal{P}_{n\mathcal{T}_j}) + d(\mathcal{P}_{n\mathcal{T}_j}, \mathbf{q}_t)\} \right\} \quad (15)$$

8: if  $\mathcal{C}(w_r, \theta_{\mathcal{P}_{n\mathcal{T}_j}}, d(\mathbf{q}_r, \mathcal{P}_{n\mathcal{T}_j}))$  exists then
9:    $\mathbf{q}_i \leftarrow \mathcal{P}_{n\mathcal{T}_j}$ , GOTO Step 1
10: else
11:   Move to node  $\mathcal{T}_j$ ,  $\mathcal{G}_r \leftarrow \mathcal{G}_{TraceBack}$ ,  $\mathbf{q}_i \leftarrow \mathcal{P}_{n\mathcal{T}_j}$ , GOTO Step 1
12: end if
13: EXIT

```

4.2. Exploration mode

The algorithm produces $\mathbf{q}_i$ to explore the entire bounded space Ω_{ws} . In Algorithm 3, $d(\mathbf{q}_{\mathcal{T}_r}, \mathcal{P}_{n\mathcal{T}_r})$ is the Euclidean distance between the node $\mathcal{T}_r$ and its n th possibility $\mathcal{P}_{n\mathcal{T}_r}$, and $d_R(\mathbf{q}_r, \mathbf{q}_{\mathcal{T}_j})$ is the

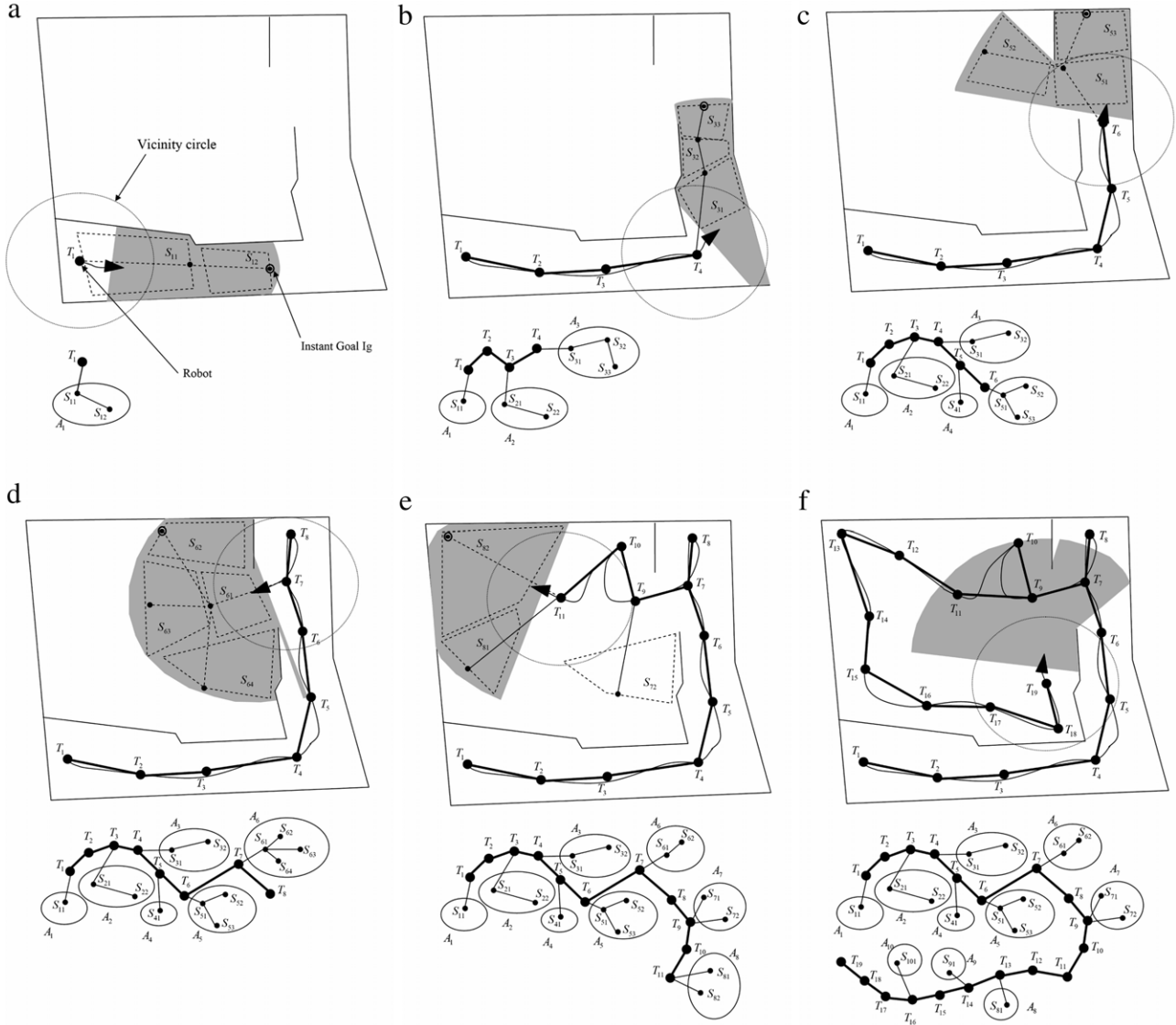


Fig. 12. Incremental construction of the HTM.

route distance between $\mathbf{q}_r$ and $\mathbf{q}_{T_j}$. Step 11 in the algorithm is a map updating step in addition to the node registration procedure mentioned in Section 3.3. If I_g represents a corner or dead end (Fig. 11), the map is incomplete. The updating process is necessary to ensure that the robot never revisits this portion of the environment.

4.3. Trace back mode

In both the path planning tasks discussed, situations arise where it is necessary to move back to a destination node T_d in $\mathcal{K}$. Instant goals I_g are generated to enable the robot to move along $\mathcal{K}$ by computing the route to T_d , $R_{r,d} = [T_r \dots T_d]$. In Algorithm 4, $d(\mathbf{q}_r, \mathbf{q}_{T_d})$ is the Euclidean distance between $\mathbf{q}_r$ and T_d and $T_v(r, d)$ is the visibility node in the route $R_{r,d}$.

4.4. Planning in a dynamic environment with localization errors

The SP²ATM algorithm can also produce a complete solution in dynamic environments through continuous replanning since the AST only registers $\mathcal{S}_{AF}$. However, if an assigned possibility $\mathcal{P}_{T_j}$ or registered node T_j is later inaccessible, i.e. the robot cannot reach the vicinity circle V_r of T_j or $\mathcal{P}_{T_j}$ within σ_g , the convergence of the algorithm may not be guaranteed. The following additional steps

are appended to the existing algorithm for planning in a dynamic environment.

- 1: **if** $\mathcal{C}(w_r, \theta_{\mathcal{P}_{T_j}}, d(\mathbf{q}_r, \mathcal{P}_{T_j}))$ does not exist **then**
- 2: **if** $|t - t_p| > t_d$ **then**
- 3: inaccessible possibility, $\mathcal{K} = \mathcal{K} \setminus \mathcal{P}_{T_j}$
- 4: **end if**
- 5: **end if**
- 6: **if** $\mathcal{G}_r = \mathcal{G}_{\text{TraceBack}}$ **then**
- 7: **if** $\mathcal{C}(w_r, \theta_{T_j}, d(\mathbf{q}_r, \mathbf{q}_{T_j}))$ does not exist **then**
- 8: **if** $|t - t_n| > t_d$ **then**
- 9: inaccessible node, $\mathcal{K} = \mathcal{K} \setminus (T_j, L_{\mathcal{K}}(r, j))$
- 10: **end if**
- 11: **end if**
- 12: **end if**
- 13: EXIT

where t_p and t_n are the times at which an inaccessible possibility and node are found. t_d is termed as the tolerance period for which the planner waits to reconfirm the inaccessibility of a node or possibility.

The incremental construction of the HTM can now be described by focussing on the task of exploration. The map building process is shown in Fig. 12. Initially the robot observes the admissible

Algorithm 3 Exploration mode

```

1: if  $\mathcal{P}_{\mathcal{T}_j} = \emptyset$ ,  $0 < j \leq N_{\mathcal{T}}$  then
2:   if  $d(\mathbf{q}_{\mathcal{T}_5}, \mathbf{q}_r) \leq \sigma_g$  then
3:     Exploration complete,  $\mathcal{G}_r \leftarrow \mathcal{G}_{\text{Brake}}$ , EXIT
4:   else
5:     Move back to start node  $\mathcal{T}_5$ ,  $\mathcal{G}_r \leftarrow \mathcal{G}_{\text{TraceBack}}$ , GOTO Step 2
6:   end if
7: end if
8: Compute nearest possibility  $\mathcal{P}_{n\mathcal{T}_r}$  satisfying the equation
   
$$\min_{0 < n < N_{\mathcal{P}_{\mathcal{T}_r}}} \{d(\mathbf{q}_{\mathcal{T}_r}, \mathcal{P}_{n\mathcal{T}_r})\} \quad (16)$$

9:  $\mathbf{q}_i \leftarrow \mathcal{P}_{n\mathcal{T}_r}$ 
10: if  $d(\mathbf{q}_r, \mathbf{q}_i) \leq \sigma_g$  then
11:   Update an end node  $\mathcal{T}_r$  in  $\mathcal{K}$ 
12: end if
13: if  $E_{\mathcal{T}_r} = E_1$  then
14:   Search and move to unexplored ( $E_2$ )  $\mathcal{T}_j$  satisfying
     
$$\min_{0 < j \leq N_{\mathcal{T}}} \{d_R(\mathbf{q}_r, \mathbf{q}_{\mathcal{T}_j})\} \quad (17)$$

15:   if  $\mathcal{T}_j$  exists then
16:      $\mathcal{G}_r \leftarrow \mathcal{G}_{\text{TraceBack}}$ , move to  $\mathcal{T}_j$ , GOTO Step 8
17:   else
18:     No unexplored ( $E_2$ ) node exists, GOTO Step 1
19:   end if
20: else
21:    $E_{\mathcal{T}_r} = E_2$ , GOTO Step 8
22: end if
23: EXIT

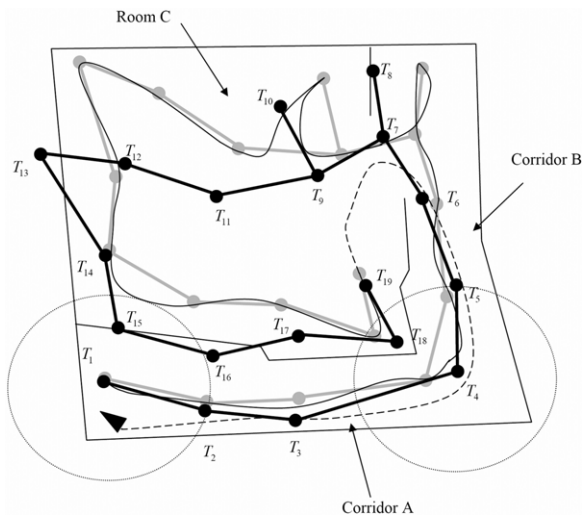
```

Algorithm 4 Trace back mode

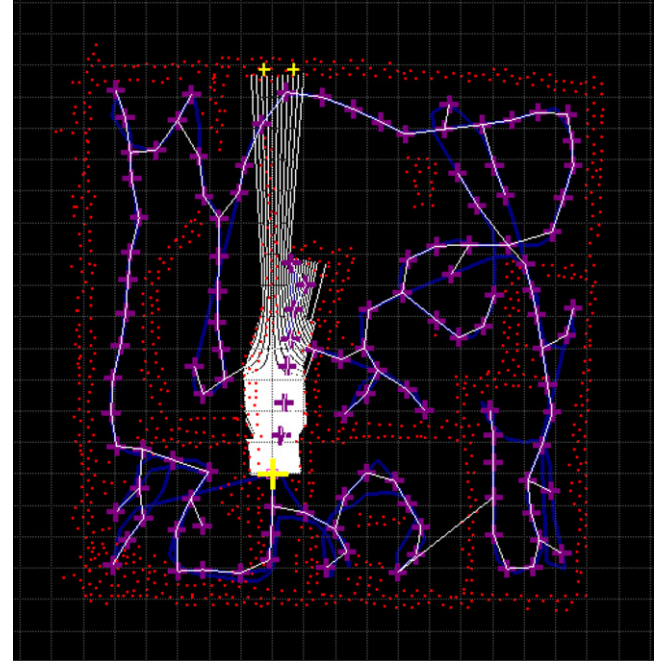
```

1: if  $d(\mathbf{q}_r, \mathbf{q}_{\mathcal{T}_d}) \leq r_t$  then
2:   Switch to previous state  $\mathcal{G}_{\text{GoalApp}}$  OR  $\mathcal{G}_{\text{Exp}}$ 
3: end if
4: if  $\mathcal{T}_v(r, d)$  exists then
5:    $\mathbf{q}_i = \mathbf{q}_{\mathcal{T}_v(r, d)}$ , GOTO Step 1
6: else
7:    $\mathbf{q}_i = \mathbf{q}_{\mathcal{T}_{r-1}}$ , GOTO Step 1
8: end if
9: EXIT

```

**Fig. 13.** Shifted HTM due to localization drift.

spaces on the right and the AST $\mathcal{A}_1$ with segments S_{11} and S_{12} are created as shown in Fig. 12(a). The initial node $\mathcal{T}_5 = \mathcal{T}_1$ is assigned with $\mathcal{A}_1$. The robot moves towards the instant goal created from leaf S_{12} in $\mathcal{A}_1$. As the robot moves, it leaves the vicinity circle of

**Fig. 14.** Map built after exploration experiment of a 289 m² area. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

$\mathcal{T}_1$ and hence places a new node $\mathcal{T}_2$. $\mathcal{T}_2$ is a dummy node which facilitates navigation since there are no new admissible spaces that can be described and S_{12} is already in the robot's vicinity. Node $\mathcal{T}_3$ is created when the new AST $\mathcal{A}_2$ is obtained. It is to be noted that the admissible space S_{12} is transferred to $\mathcal{A}_2$ as S_{21} since $\mathcal{T}_3$ is closer to it than $\mathcal{T}_1$ as per the pre-assigned possibility rearrangement criterion (Section 3.2.4). The HTM grows in upper and lower layers when new areas are perceived as the robot moves towards the instant goal obtained at every sample time. It can be seen from the robot trace that no excessive time is spent in a single location and that the motion is smooth. The robot completes exploring the area in Fig. 12(f). The inaccessibility problem is less likely to happen due to the localization errors which are bounded as per our assumption.

Lemma 1. Assume that due to the dead reckoning error, the map as perceived by the robot is shifted both in translational and rotational manner. The actual map built under accurate and inaccurate localization assumption is shown light grey and black, respectively, in Fig. 13. The HTM is able to assist the planner in promoting reliable robot navigation under bounded drift assumption.

Proof. Consider the room C. After the exploration is complete, the robot tries to move back to the starting node through route $R_{19,1}$. The drift in the area is always limited when any node in the room is compared with its entrance node $\mathcal{T}_7$. $\mathcal{T}_7$ is the visibility node since it is completely visible from the robot location node $\mathcal{T}_r = \mathcal{T}_{19}$ and is farthest in the route. Therefore, the robot does not have any problems tracing back towards $\mathcal{T}_7$. The trace back route is shown as a dotted curve. While it moves towards $\mathcal{T}_7$, $\mathcal{T}_6$ becomes visible and then $\mathcal{T}_4$ in corridor B. Since none of the nodes in corridor A are visible, the robot moves until it is sufficiently in the vicinity of $\mathcal{T}_r = \mathcal{T}_4$ and then moves to $\mathcal{T}_{r-1}$ i.e. $\mathcal{T}_3$ in $R_{19,1}$. The process repeats until the robot comes to the vicinity of $\mathcal{T}_1$. However the drift assumed here is large and may cause the navigation to fail if any more localization error accumulates. Consider node $\mathcal{T}_{16}$ which projects out from room C to corridor A. The robot moves close to the wall to reach $\mathcal{T}_1$ during its trace back and always remains closer to $\mathcal{T}_1$, $\mathcal{T}_2$ or $\mathcal{T}_3$ when compared to $\mathcal{T}_{16}$.

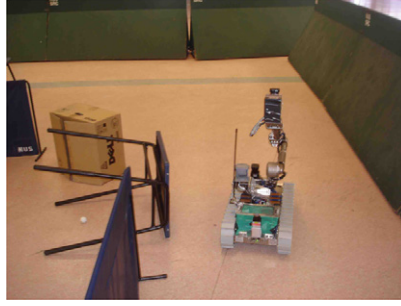
(a) $t = 4$ s.(b) $t = 8$ s.(c) $t = 14$ s.(d) $t = 18$ s.(e) $t = 26$ s.(f) $t = 37$ s.(g) $t = 47$ s.(h) $t = 57$ s.(i) $t = 64$ s.

Fig. 15. Snapshots showing dynamic nature of the exploration experiment in a 150 m² area.

If the robot computes $\mathcal{T}_r = \mathcal{T}_{16}$, the planner assumes inaccessible node condition, keeps on building the map and does not converge. Hence, under bounded error assumption, if the motion control follows the planner commands and does not overshoot, the robot can navigate reliably in the environment. $\square$

Theorem 1. The SP²ATM algorithm is complete.

(a) Assuming that Ω_{ws} is bounded, the robot completes exploratory path planning by visiting the entire $\mathcal{S}_{AF}$ to reach its final configuration $\mathcal{T}_s$ in finite time.

(b) The robot reaches its final configuration $\mathbf{q}_r = \mathbf{q}_t$ for goal oriented path planning in finite time, if a path to $\mathbf{q}_t \in \mathcal{S}_{AF}$ exists.

Proof. (a) At each step, the AST $\mathcal{A}_k$ is generated for the newly perceived admissible space during robot motion. The topology map $\mathcal{M}$ is updated by registering new topology node $\mathcal{T}_r$ and assigning possibilities $\mathcal{P}_{\mathcal{T}_r}$ extracted from the AST $\mathcal{A}_k$. Each possibility in $\mathcal{M}$ represents a portion of the whole admissible space in Ω_{ws} and implies potential direction to explore. The robot continues its exploration towards a sequence of instant goals until another new admissible space is perceived or all the possibilities are visited in terms of the distance to robot position $\mathbf{q}_r$. This enables that the robot always moves towards newly perceived areas while constructing new ASTs. Since Ω_{ws} is bounded, the admissible

space $\mathcal{S}_{AF}$ which is divided into subspaces by finite number of possibilities, is bounded. Therefore, after several steps, if $\mathcal{P} = \emptyset$, i.e. the SP²ATM algorithm has guided the robot to visit all the possibilities in sequence and move through the entire $\mathcal{S}_{AF}$ in Ω_{ws} , the exploratory path planning is completed in finite time.

(b) For bounded Ω_{ws} , as stated in (a), since the algorithm is capable of moving through the whole $\mathcal{S}_{AF}$, it is guaranteed that the robot will always reach the target $\mathbf{q}_t \in \mathcal{S}_{AF}$. If Ω_{ws} is unbounded, since the possibility generation for goal oriented path planning favors the portion in the AST closer to $\mathbf{q}_t$ and the nearest possibility to $\mathbf{q}_t$ among all the existing possibilities will be selected as the instant goal at each step, it ensures that the robot $\mathbf{q}_r$ will always tend to approach $\mathbf{q}_t$ and not diverge until the robot completes the task. $\square$

5. Experimental results

The experiments were conducted on our robot X1 equipped with a Hokuyo UTM-30LX laser scanner ($\gamma_h = \pi$, $d_{\max} = 30$ m) interfaced to a 1.6 GHz Pentium M (1 GB RAM) computer. For the localization of the robot, we used a simple system consisting of the onboard odometry with a single axis fiber optic gyroscope [33], which made our bounded error assumption possible. The sampling time t_s of the navigation system was set to 0.5 s and the robot forward and angular velocity were bounded to 0.6 m/s and

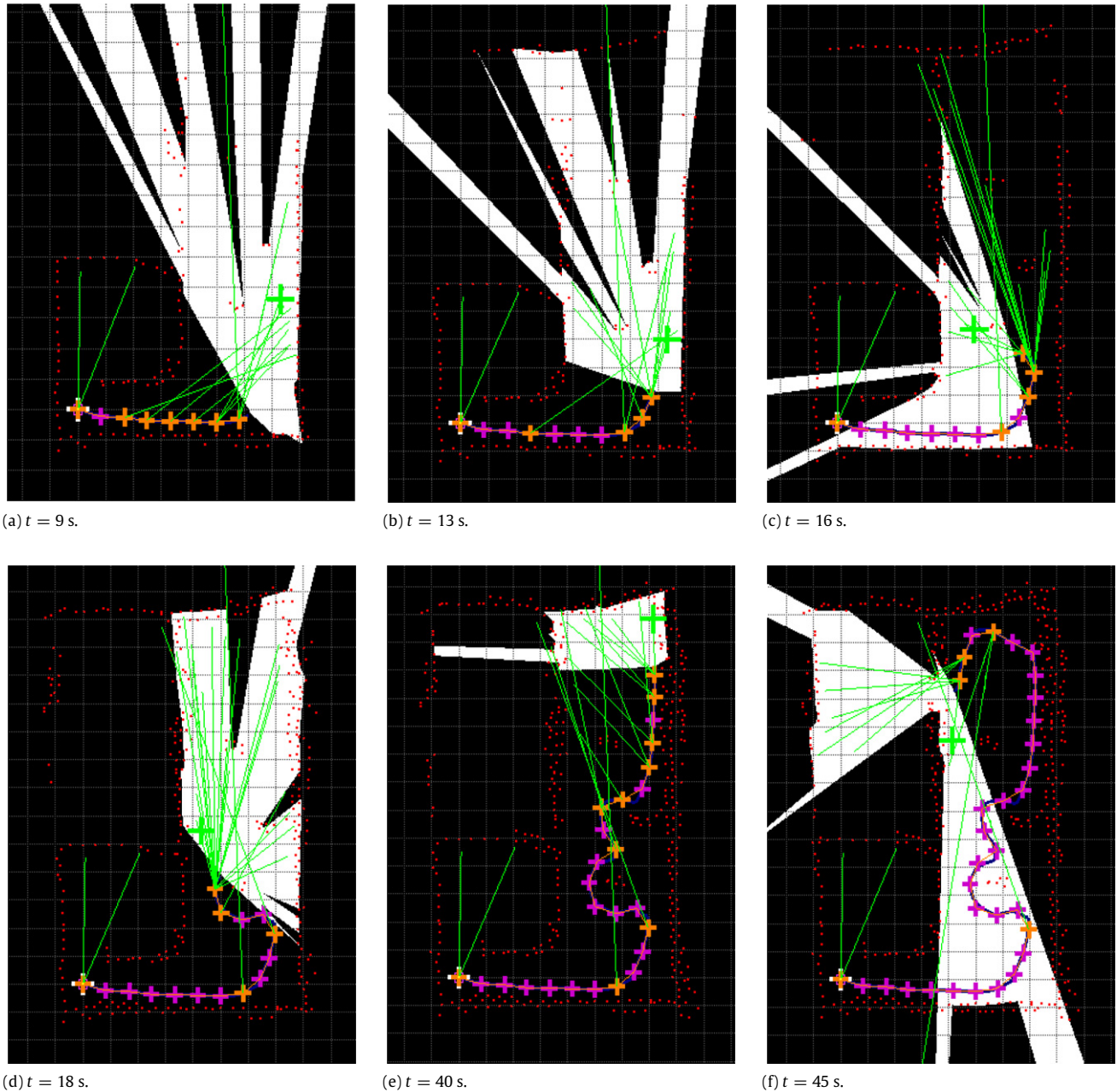


Fig. 16. Map building process showing dynamic nature of the exploration experiment in a 150 m² area. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

1.0 rad/s, respectively. The computation time required for the module for every sampling time was found to be as low as 0.12 s and was seen to rise up to 0.36 s in worst cases when the area perceived by the laser was large. The actual base width of our robot is $w_r = 0.5$ m and hence, w_v was chosen to be 0.8 m. The SP²ATM algorithm steered X1 while participating in the TechX Challenge competition in Singapore, 2008.¹

5.1. Exploratory path planning

An exploration test was conducted in a complex, static indoor environment of area 289 m² with obstacles. The grid in grey color

shows the scale of the map with each square representing an area of 1 m². The whole map explored is shown in Fig. 14. The total time taken for coverage was only 10 min 40 s. In the figure, crosses in magenta are the explored nodes having connections (white) in between them. The start node $\mathcal{T}_s$ is shown by a yellow cross from which the laser scan (white) lines spread out after mission completion. The robot's path traveled is also shown in blue. The time taken depends on the complexity and area of the environment. For a real experiment, since the complexity is unknown, minimum distance $d_{\mathcal{T}}$ was set to be 0.8 m, which is small enough so that the possibilities perceived could be directly accessible since no local planner was used. This is also necessary for the robot to trace back through the known environment since the navigation is purely topology based. If only map building is of prime concern, a map with widely spaced nodes could be generated by increasing $d_{\mathcal{T}}$.

¹ TechX Challenge is a robot competition organized by DSTA, Singapore's national authority for defence science and technology. For more information, refer to <http://www.dsta.gov.sg/index.php/TechX-Challenge/>.

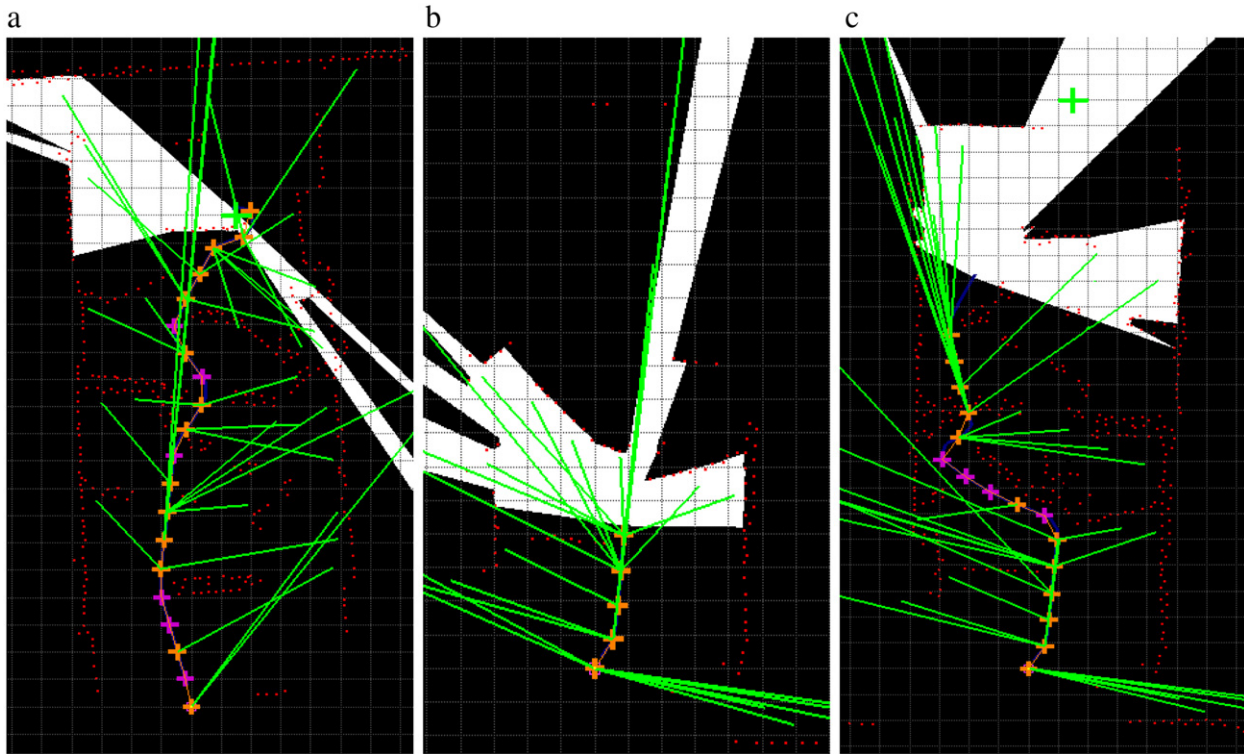


Fig. 17. Map built during goal oriented path planning in a static and dynamic environment. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

In order to examine the behavior in a dynamic environment, the robot was put to test in a small area of 150 m² with a person moving around the bounded region. A portion of the environment was also opened up for the robot to place a new possibility outside the bounded area and was closed later to observe the behavior. The portion of the experiment which adapts to the situation is shown with the sequence of events in Fig. 15 and the map building process in Fig. 16. In the map, the explored nodes are shown in magenta, unexplored nodes and edges in orange, line extending towards a possibility in green and the instant goal, a green cross. $\mathcal{T}_s$ is shown by a big white cross on a square outlined in magenta. The obstacles seen are shown as red dots. The robot initially observes the room in the front and the corridor towards the right (Fig. 15(a), (b), Fig. 16(a), (b)). It finds two possibilities in the room and several of them in the corridor. These are associated with nodes in the topology map as per the criterion in Section 3.2. As the robot moves, the instant goal shifts and the robot moves further. It can be seen that the assigned possibilities are removed and shifted to forward nodes. In Fig. 16(a), a portion of the environment is opened up and the laser rays fall out of the boundary. This enables the planner to place a possibility outside the barrier. In Fig. 16(c), this area is closed up. However the possibility remains in memory until Fig. 16(f) in which, the registered possibility outside the boundary is chosen and removed from the memory after a timeout of $t_d = 5$ s. The exploration of the area was complete in 4 min 42 s.

5.2. Goal oriented path planning

For testing the performance of the goal approaching mode in a static environment, a goal was placed 18 m away from the robot at an angle 5° from the global frame y axis. Nodes were registered when new possibilities are found while focussing only on the goal. This is the reason why a few possibilities can be seen left behind on

either side of the path traveled. It deals well with close obstacles without relying on any collision avoidance layer. The map built is shown in Fig. 17(a). The path planning was completed in 59 s. The path of the robot can be seen from the trail of nodes and can be observed that it is optimal.

A final experiment was done to test how the goal approaching algorithm would behave in a dynamic environment. The same coordinates were given in a similar, but a different setting. In Fig. 17(b), the robot sees the goal directly and places the instant goal on it. However, the path is later blocked and when the visibility to the instant goal fails, the robot waits for time t_d , reconfirms it and deletes the possibility from memory. The robot then plans the next best path towards the goal as shown in Fig. 17(c) completing the task in 1 min 33 s.

6. Conclusion

This paper has formulated a new approach, Simultaneous Path Planning and Topological Mapping (SP²ATM), to solve the problem of path planning in unknown environments, assisted by an incrementally and concurrently built topological map. First, a representation of the environment which demands minimal storage requirements was made possible through a topological approach. The traversable regions in the environment, as perceived by a range sensor, were modeled as an Admissible Space Tree (AST). The global topology of the environment was represented by a Hierarchical Topological Map (HTM) comprised of an undirected graph with each node describing the perceived AST from its location. The HTM provides the least information for a robot's path planning and the technique is advantageous in both storage and processing time. Secondly, the representation was shown to cope with sensor uncertainties and robot positioning errors. The bounded error assumption for localization was found to hold true not only due to the reliable localization system, but also

since the tasks could be completed in a small time duration. The attempt is only to make the system simple and does not rule out the need of a SLAM module which is inevitable in a very large area for long durations. Finally, a map-based path planner which guides the robot along an optimal path to move towards a known goal for goal oriented path planning and visit all spaces in an environment for exploratory path planning was proposed.

SP²ATM has been implemented on a differentially driven mobile robot equipped with a single laser scanner, encoders and a gyroscope. Since the non-semantic method does not rely on any geometric information specific to an environment, the planner enables the robot to navigate anywhere in the realistic world. The path planned towards the goal was found to be optimal and the task of exploration consumed a short duration in the complex environment and dynamic environment by introducing a wait strategy. The algorithm, however, may consume more time if the sensor range and field of view were very less compared to the environment at hand and the size of AST is relatively small. Additionally, even though the experiments have been conducted without a local planner, the SP²ATM is not a completely robust solution by itself in highly cluttered and constrained environments. The robot motion could be very slow and would require the help of a simple collision avoidance module like a local potential function for faster convergence. One interesting research direction is to consider the vehicle shape, kinematics, and dynamics of the mobile robot in a collision avoidance layer [29,34] which could greatly improve the exploration of the local free space.

References

- [1] H. Choset, K. Lynch, S. Hutchinson, G. Kantor, W. Burgard, L. Kavraki, S. Thrun, Principles of Robot Motion: Theory, Algorithms, and Implementation, MIT Press, 2005.
- [2] S.S. Ge, F. Lewis, Autonomous Mobile Robots: Sensing, Control, Decision Making and Applications, CRC Press, Taylor and Francis Group, Boca Raton, FL, 2006.
- [3] H. Choset, J. Burdick, Sensor based planning, part I: the generalized Voronoi graph, in: Proceedings—IEEE International Conference on Robotics and Automation, vol. 2, 1995.
- [4] J. Leonard, H.F. Durrant-Whyte, Mobile robot localization by tracking geometric beacons, in: Proceedings of the Ninth International Symposium on Robotics Research, vol. 7, no. 3, 1991, pp. 376–382.
- [5] H. Gonzalez-Banos, J. Latombe, Navigation strategies for exploring indoor environments, The International Journal of Robotics Research 21 (10–11) (2002) 829–848.
- [6] H. Feder, J. Leonard, C. Smith, Adaptive mobile robot navigation and mapping, The International Journal of Robotics Research 18 (7) (1999) 650.
- [7] B. Tovar, L. Muñoz-Gómez, R. Murrieta-Cid, M. Alencastre-Miranda, R. Monroy, S. Hutchinson, Planning exploration strategies for simultaneous localization and mapping, Robotics and Autonomous Systems 54 (4) (2006) 314–331.
- [8] V. Lumelsky, A.A. Stepanov, Path-planning strategies for a point mobile automaton moving amidst unknown obstacles of arbitrary shape, Algorithmica 2 (1987) 403–430.
- [9] J. Borenstein, Y. Koren, The vector field histogram-fast obstacle avoidance for mobile robots, IEEE Transactions on Robotics and Automation 7 (3) (1991) 278–288.
- [10] S.S. Ge, Y.J. Cui, New potential functions for mobile robot path planning, IEEE Transactions on Robotics and Automation 16 (5) (2000) 615–620.
- [11] H. Choset, Coverage for robotics—a survey of recent results, Annals of Mathematics and Artificial Intelligence 31 (2001) 113–126.
- [12] S. Thrun, Learning metric-topological maps for indoor mobile robot navigation, Artificial Intelligence 99 (1) (1998) 21–71.
- [13] A. Elfes, Sonar-based real-world mapping and navigation, IEEE Journal of Robotics and Automation 3 (3) (1987) 249–265.
- [14] X.C. Lai, S.S. Ge, A. Al Mamun, Hierarchical incremental path and motion planning for differential steering mobile robots considering accelerations for forward and turning speeds, IEEE Transactions on Systems, Man and Cybernetics, Part B (Cybernetics) 37 (6) (2007) 1541–1554.
- [15] E. Remolina, B. Kuipers, Towards a general theory of topological maps, Artificial Intelligence 152 (2004) 47–104.
- [16] S. Thrun, A. Bücken, Integrating grid-based and topological maps for mobile robot navigation, in: Proceedings of the National Conference on Artificial Intelligence, 1996, pp. 944–951.
- [17] B. Kuipers, Y.-T. Byun, A robot exploration and mapping strategy based on a semantic hierarchy of spatial representations, Robotics and Autonomous Systems 8 (1–2) (1991) 47–63.
- [18] G. Dudek, M. Jenkin, E. Milios, D. Wilkes, Robotic exploration as graph construction, IEEE Transactions on Robotics and Automation 7 (6) (1991) 859–865.
- [19] G. Dudek, M. Jenkin, E. Milios, D. Wilkes, On map exploration, validation, and self-location in a graph-like world, Robotics and Autonomous Systems 22 (2) (1997) 159–178.
- [20] V.J. Lumelsky, S. Mukhopadhyay, K. Sun, Dynamic path planning in sensor-based terrain acquisition, IEEE Transactions on Robotics and Automation 6 (4) (1990) 462–472.
- [21] M. Mataric, Integration of representation into goal-driven behavior-based robots, IEEE Transactions on Robotics and Automation 8 (3) (1992) 304–312.
- [22] B.S. Ryu, H.S. Yang, Integration of reactive behaviors and enhanced topological map for robust mobile robot navigation, IEEE Transactions on Systems, Man & Cybernetics, Part A (Systems & Humans) 29 (5) (1999) 474–485.
- [23] G. Foux, M. Heymann, A. Bruckstein, Two-dimensional robot navigation among unknown stationary polygonal obstacles, IEEE Transactions on Robotics and Automation 9 (1) (1993) 96–102.
- [24] M. Barbehenn, S. Hutchinson, Efficient search and hierarchical motion planning by dynamically maintaining single-source shortest path trees, IEEE Transactions on Robotics and Automation 11 (2) (1995) 198–214.
- [25] H. Choset, J. Burdick, Sensor-based exploration: the hierarchical generalized Voronoi graph, The International Journal of Robotics Research 19 (2) (2000) 96–125.
- [26] H. Choset, S. Walker, K. Eiamsa-Ard, J. Burdick, Sensor-based exploration: incremental construction of the hierarchical generalized Voronoi graph, The International Journal of Robotics Research 19 (2) (2000) 126–148.
- [27] B. Tovar, R. Murrieta-Cid, S.M. LaValle, Distance-optimal navigation in an unknown environment without sensing distances, IEEE Transactions on Robotics and Automation 23 (3) (2007) 506–518.
- [28] J. Minguez, L. Montano, Nearness diagram (ND) navigation: collision avoidance in troublesome scenarios, IEEE Transactions on Robotics and Automation 20 (1) (2004) 45–59.
- [29] J. Blanco, J. González, J. Fernández-Madrigal, Extending obstacle avoidance methods through multiple parameter-space transformations, Autonomous Robots 24 (1) (2008) 29–48.
- [30] E. Acar, H. Choset, J. Lee, Sensor-based coverage with extended range detectors, IEEE Transactions on Robotics 22 (1) (2006) 189–198.
- [31] S.S. Ge, X.C. Lai, A. Al Mamun, Boundary following and globally convergent path planning using instant goals, IEEE Transactions on Systems, Man and Cybernetics, Part B (Cybernetics) 35 (2) (2005) 240–254.
- [32] R.E. Bellman, Dynamic Programming, Princeton Univ. Press, Princeton, NJ, 1957.
- [33] H. Chung, L. Ojeda, J. Borenstein, Accurate mobile robot dead-reckoning with a precision-calibrated fiber-optic gyroscope, IEEE Transactions on Robotics and Automation 17 (1) (2001) 80–84.
- [34] J. Minguez, L. Montano, Extending collision avoidance methods to consider the vehicle shape, kinematics, and dynamics of a mobile robot, IEEE Transactions on Robotics 25 (2) (2000) 367–381.



Shuzhi Sam Ge received the B.Sc. degree from Beijing University of Aeronautics and Astronautics (BUAA), Beijing, China, in 1986, and the Ph.D. degree from Imperial College of Science, Technology and Medicine, University of London, London, UK, in 1993.

He is founding Director of the Social Robotics Lab, Interactive Digital Media Institute, and Professor with the Department of Electrical and Computer Engineering, the National University of Singapore, Singapore and the Institute of Intelligent Systems and Information Technology, University of Electronic Science and Technology of China, Chengdu, China. He has (co-)authored three books, edited a book, and published over 150 international journal papers. He is the Editor-in-Chief of the International Journal of Social Robotics. He has served/been serving as an Associate Editor for a number of flagship journals. He also serves as an Editor of the Taylor & Francis Automation and Control Engineering Series. He has been serving as Vice President of Technical Activities, 2009–2010, IEEE Control Systems Society. His current research interests include social robotics, multimedia fusion, adaptive control, and intelligent systems.



Qun Zhang received the B.Eng. degree in Electronics & Information Engineering from Chang'an University, Xi'an, China, in 2005 and his M.Eng. degree in Electrical Engineering from Xi'an Jiaotong University, Xi'an, China, in 2008. He is currently working towards his Ph.D. degree at the NUS Graduate School for Integrative Sciences and Engineering (NGS), National University of Singapore (NUS), Singapore. His research interests include autonomous robot navigation and machine learning.



Aswin Thomas Abraham received the B.Eng. degree in Electronics & Communication Engineering from Mahatma Gandhi University, Kerala, India, in 2006 and his M.Sc. degree in Mechatronics from the National University of Singapore (NUS), Singapore, in 2008. He is currently working towards his Ph.D. degree at the NUS Graduate School for Integrative Sciences and Engineering (NGS), National University of Singapore (NUS), Singapore. His research interests include autonomous robot navigation, machine learning and perception.



Brice Rebsamen received the M.S. degree in electronics from ENSEIRB, France, in 2002, the M.S. degree in cognitive sciences from INPG, France, in 2003, and the Ph.D. degree from the National University of Singapore (NUS), in 2010. He has been working as a Research Scientist at NUS since January 2008. His research interests include autonomous robotics, brain–computer interfaces, and cognitive sciences. He is particularly interested in medical and rehabilitation applications.